running time is `O(D*logD(n))`, confirming what we have discussed above and summarized in table 2.7. Not surprisingly, the `_push_down` helper method has an identical distribution.

If we look at listings 2.7 and 2.4, sections 2.6.2 and 2.6.4, it's clear how `pushDown` is the hearth of method `top`, and in turn the largest chunk of work for `pushDown` is the method that at each heap's level retrieves the child of current node, we called it `highestPriorityChild`[28].

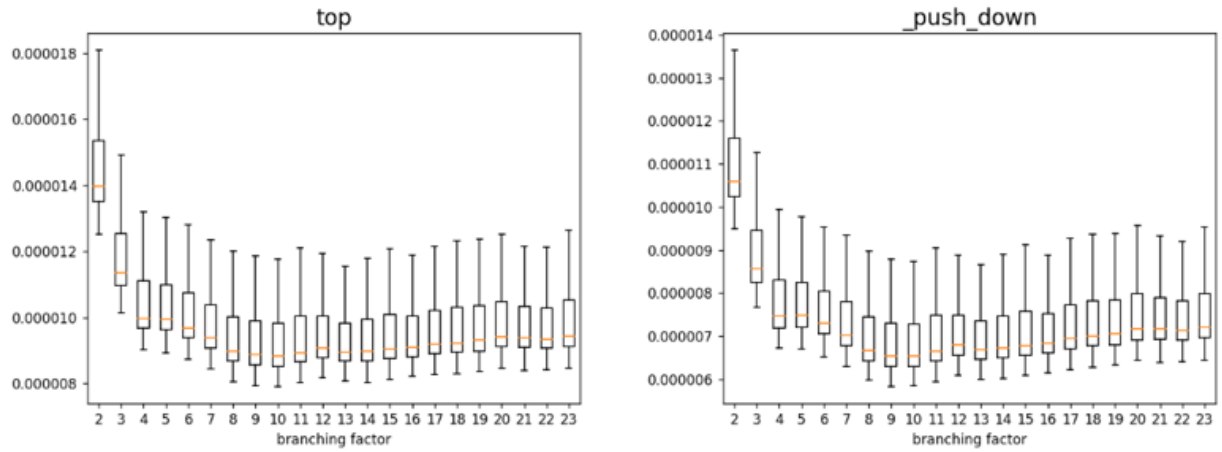


Figure 2.23 The distribution of recorded running times per-call for `top` and `_push_down`.

And if then we look at the running time *per-call* for `_highest_priority_child` (figure 2.24.A), we have a confirmation that our profiling does make sense, as the running time per-call increases with the branching factor: the largest `D` is, the longest the list of children for each node, which this method need to traverse entirely in order to find which tree branch should be traversed next.